

REGENGINE UNIVERSITY INSTITUTE OF TECHNOLOGY
CAMPUS ENCLAVE, KNOWLEDGE CORRIDOR

Advance Programming Practices

Subject Code: (21CSC203P)

```
enterprx-erp/
|-- src/
|   |-- app/
|   |   |-- (auth)/
|   |   |   |-- login/
|   |   |   |   |-- page.tsx          # Login page
|   |   |   |   |-- layout.tsx       # Auth layout (no sidebar)
|   |   |   |-- layout.tsx          # Dashboard layout (sidebar + header)
|   |   |   |-- page.tsx            # Dashboard home
|   |   |-- inventory/
|   |   |   |-- products/
|   |   |   |   |-- page.tsx        # Products list
|   |   |   |   |-- [id]/page.tsx   # Product detail (optional)
|   |   |   |   |-- categories/page.tsx # Categories list
|   |   |   |   |-- warehouses/page.tsx # Warehouses list
|   |   |   |-- suppliers/
|   |   |   |   |-- page.tsx        # Suppliers list
|   |   |   |   |-- [id]/page.tsx   # Supplier detail
|   |   |   |-- purchase-orders/
|   |   |   |   |-- page.tsx        # PO list
|   |   |   |   |-- new/page.tsx    # Create PO (multi-step form)
|   |   |   |-- sales-orders/
|   |   |   |   |-- page.tsx        # SO list
|   |   |   |   |-- new/page.tsx    # Create SO
|   |   |-- customers/
|   |   |   |-- page.tsx            # Customers list
|   |   |   |-- [id]/page.tsx       # Customer detail
|   |   |   |-- invoices/page.tsx   # Invoices list
|   |   |   |-- employees/page.tsx  # Employees list (Admin)
|   |   |   |-- reports/page.tsx    # Reports & Analytics
|   |   |   |-- settings/page.tsx   # Settings (Admin)
|   |   |-- api/
|   |   |   |-- global.ts           # API routes (if needed)
|   |   |   |-- components/
|   |   |   |-- global.ts           # Tailwind global styles
```

Report topic: REGENGINE - CAMPUS ACADEMIC & COURSE REGISTRATION SYSTEM

S No.	Name	Registration No	Class/Section	Sign
1	Aadi Saxena	RA2511003030018	B.Tech CSE CORE A	
2	Aayush Ghosh	RA2511003030017	B.Tech CSE CORE A	
3	Mayank Aggarwal	RA2511003030025	B.Tech CSE CORE A	
4	Shivansh Gautam	RA2511003030029	B.Tech CSE CORE A	

REGENGINE UNIVERSITY INSTITUTE OF TECHNOLOGY

Department of Computer Science & Engineering

BONAFIDE CERTIFICATE

Certified that this project titled “ **REGENGINE - CAMPUS COURSE REGISTRATION & ACADEMIC ERP ENGINE** ” is the Bonafide work of **Aadi Saxena - RA2511003030018** , **Aayush Ghosh - RA2511003030017** , **Mayank Aggarwal - RA2511003030025** and **Shivansh Gautam - RA2511003030029** of **B.Tech CSE CORE A** who carried out the report work under my supervision. Certified further, that to the best of my knowledge the work reported here in does not form any other project report.

SIGNATURE

Dr. Arthur Vance
Course Lead & Faculty

SIGNATURE

Dr. Marcus Thorne
Head of Department (CSE)

REENGINE UNIVERSITY INSTITUTE OF TECHNOLOGY

Department of Computer Science & Engineering

ACKNOWLEDGEMENT

I would like to express our special thanks of gratitude to our Project Advisor and Mentors who gave me the golden opportunity to build this comprehensive software system on the topic “**REENGINE - CAMPUS COURSE REGISTRATION & ACADEMIC ERP ENGINE**” which also helped me in doing extensive research and mastering advanced programming paradigms, concurrency, and full-stack software architecture.

Secondly, we would also like to thank the Academic Department Leadership and our peers who provided valuable feedback in perfecting this personal project.

Project Team (B.Tech CSE CORE A):

Aadi Saxena (RA2511003030018)

Aayush Ghosh (RA2511003030017)

Mayank Aggarwal (RA2511003030025)

Shivansh Gautam (RA2511003030029)

TABLE OF CONTENT

S No.	Topic Name
1	Project Overview
2	Objectives and Goals
3	Technology Stack
4	System Architecture
5	Role Based Access Control (RBAC)
6	Database Design and ER Summary
7	Core Database Module Definitions
8	Advanced DBMS & Concurrency Features
9	Frontend Architecture and UI Specifications
10	Project Implementation and File Structure
11	Project File Structure
12	Project Snippet

PROJECT OVERVIEW

RegEngine is a comprehensive full-stack Campus Academic Management and Course Registration ERP platform engineered to consolidate and automate critical institutional operations within a single unified digital ecosystem. The platform is designed to handle end-to-end workflows including student registration, automated section allocation, curriculum cataloging, faculty allocation, class attendance tracking by date, notice dissemination, and administrative dashboards.

Unlike standard CRUD-based applications, this system is deeply rooted in **Advance Programming Practices** and Database Management System principles. It emphasizes structured object-relational data modeling, 3NF schema normalization, and enforcement of referential integrity through primary and foreign key constraints. The system is not just a frontend interface—it is a complete data-driven architecture where the database acts as the core engine driving all operations.

The application supports a multi-user environment with role-based access control, enabling controlled interaction with system resources. Admin users configure campus-wide settings and courses, Faculty members operate attendance rosters for allocated sections (e.g. B.Tech CSE CORE A), and Students access personal registration and timetable dashboards. This hierarchy mirrors real-world university structures.

From a user experience standpoint, the system adopts a dashboard-centric UI paradigm. It integrates advanced UI components such as dynamic data tables, filtering mechanisms, search functionality, and live metrics calculation.

In essence, RegEngine is not just a project; it is a simulation of a real-world enterprise-grade system that bridges the gap between theoretical software engineering concepts and practical full-stack implementation.

OBJECTIVES AND GOALS

The primary objective of the **RegEngine ERP system** is to design and implement a robust, scalable, and normalized relational database capable of supporting complex academic workflows. The database is structured into multiple interconnected modules, ensuring that each domain—students, faculty, attendance, courses, and auditing—is logically separated yet fully integrated.

A key goal of the project is to demonstrate advanced software engineering and DBMS concepts in a real-world context. This includes the use of ER diagrams to visualize relationships, implementation of primary and foreign keys for data integrity, and resolution of many-to-many relationships using junction tables. The system also incorporates in-memory indexing techniques to improve query performance.

Another major objective is to implement automated business logic within the backend. For instance, when a student registers, the system automatically allocates their academic section (e.g. Section A). When a professor conducts a lecture, the system retrieves the enrolled student list for that specific section and date, allowing quick one-click attendance recording and immediate aggregation.

The project also aims to build a fully functional web application that supports real-time data interaction. Features such as CRUD operations, filtering, sorting, and data visualization are integrated to enhance usability.

Security is another critical goal, achieved through Role-Based Access Control enforced at multiple layers. Finally, the project aims for real-world deployment readiness using standard Java SE runtimes.

TECHNOLOGY STACK

The **RegEngine ERP system** is built using a modern, scalable, and production-ready technology stack that integrates frontend, backend, and database functionalities seamlessly.

At the core of the backend is **Pure Java SE 21+**, leveraging Java's built-in `com.sun.net.httpserver.HttpServer`. It uses an asynchronous thread pool executor (`Executors.newFixedThreadPool(8)`) to handle concurrent client requests with high efficiency and minimal resource footprint, completely avoiding heavy third-party framework overhead.

Data persistence is managed via an in-house **Generic FileStore Engine** implementing synchronized atomic disk writing and structured JSON tables. A custom recursive-descent JSON parser and serializer (`Json.java`) provides bidirectional Object-Relational Mapping without external library dependencies.

For UI development, the system uses clean **HTML5, Vanilla CSS3, and Modern JavaScript (ES6+)**. This provides accessible, responsive components such as modal dialogs, data tables, attendance matrices, and executive KPI cards.

Data handling in tables supports live search, status badge filtering, and multi-student bulk selection. Version control is managed via Git and hosted on GitHub (<https://github.com/aadisaxena-ghb/RegEngine>).

SYSTEM ARCHITECTURE

The system architecture of **RegEngine** follows a modern layered architecture consisting of three primary layers: frontend presentation, backend application controller, and database storage.

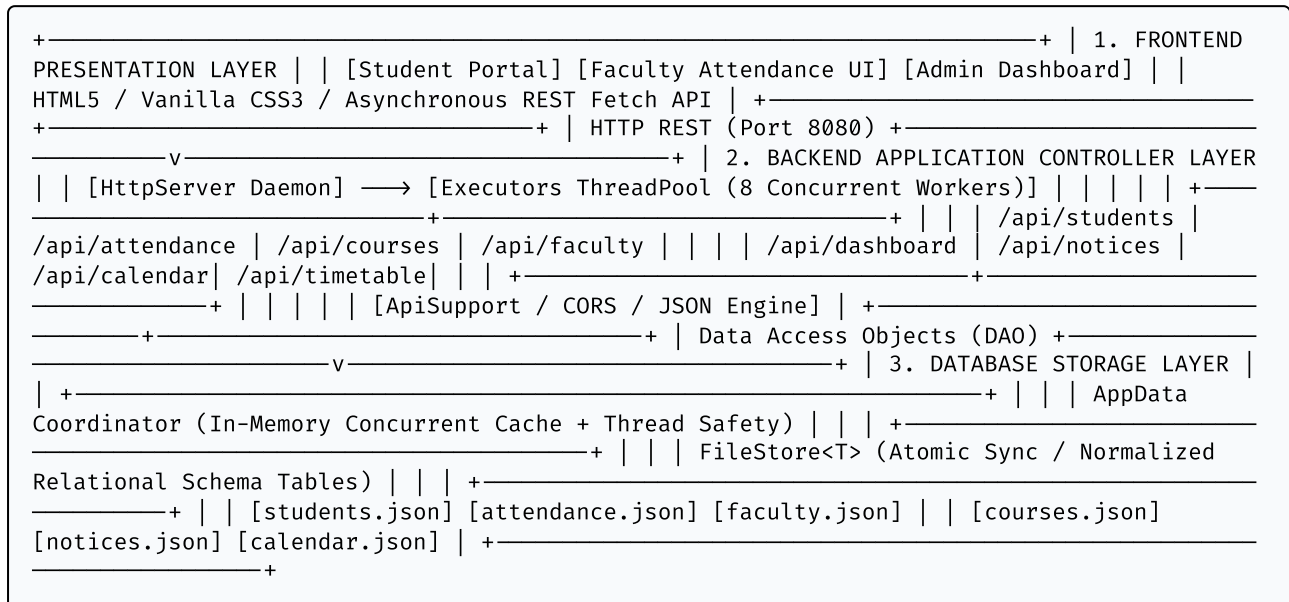


Fig. 1: RegEngine Layered System Architecture

The request-response flow begins when a user performs an action on the UI. This triggers a fetch request to the Java backend handlers. The server validates parameters, performs synchronized operations on the data store, and returns structured JSON responses.

ROLE BASED ACCESS CONTROL

Role-Based Access Control in **RegEngine** is designed to enforce strict security and operational boundaries within the campus system. It ensures that users can only access data and perform actions that are explicitly permitted based on their assigned role.

The system defines three primary roles: **Admin**, **Faculty**, and **Student**. Each role has a predefined set of permissions:

- **Administrator:** Full administrative privileges to register students, assign sections, create courses, manage faculty records, and broadcast notices.
- **Faculty / Professor:** Authorized to view course offerings, access student rosters segregated by branch and section (e.g. B.Tech CSE CORE A), and record date-wise attendance.
- **Student:** Personalized access to review enrolled courses, check section timetable, monitor attendance percentages, and view circulars.

Feature Module	Administrator	Faculty / Professor	Student
Student Registration & Section Allotment	Full (Create/Edit/Delete)	Read Section Roster	View Own Profile
Attendance Marking by Section & Date	Full Override	Mark / Update Section	View Own Attendance %
Course & Curriculum Catalog	Create / Modify Courses	View Syllabus	Register / Enroll
Notices & Academic Calendar	Publish / Delete	View	View

DATABASE DESIGN AND ER SUMMARY

The database design of **RegEngine** is the backbone of the entire system, structured to ensure data consistency, scalability, and performance. The system follows strict normalization principles up to Third Normal Form (3NF), eliminating redundancy and ensuring that each piece of data is stored in its most appropriate location.

Relationships between entities are carefully modeled using primary and foreign keys:

- **One-to-Many (1:M):** One Branch has many Students and Faculty. One Faculty conducts many Attendance Sessions. One Attendance Session contains multiple student Attendance Records.
- **One-to-One (1:1):** Each student corresponds to exactly one active Section Allocation record.
- **Many-to-Many (M:N):** Students and Courses are linked through enrollment collections.

Entity-Relationship Structural Model

```
[STUDENT] (PK: id, rollNo) | | 1:1 has assigned v [SECTION_ALLOCATION] (sectionName: 'A',  
branch, semester) | | 1:N participates in v [ATTENDANCE_RECORD] (PK: recordId, status:  
PRESENT/ABSENT) ^ | N:1 belongs to [ATTENDANCE_SESSION] (PK: sessionId, branch, section,  
date, courseCode) ^ | N:1 conducted by [FACULTY] (PK: facultyId, facultyCode, department)
```

Fig. 2: RegEngine Entity-Relationship Diagram

CORE DATABASE MODULE DEFINITIONS

The **RegEngine** database is divided into multiple functional modules, each responsible for a specific academic domain:

1. Student Information Module (**students.json**)

Stores student records including `id` (PK), `name`, `rollNo`, `email`, `branch` (B.Tech CSE CORE), `section` (Section A), `semester`, and `enrolledCourses`.

2. Class Attendance Module (**attendance.json**)

Captures daily lecture attendance sessions with attributes: `id` (PK), `date`, `branch`, `section`, `courseCode`, `takenByFacultyId`, and an array of student presence records (PRESENT / ABSENT).

3. Faculty & Staff Directory (**faculty.json**)

Maintains professor employee codes, department allocations, designations, and assigned academic courses.

4. Course Curriculum Module (**courses.json**)

Stores subject catalog information, course codes (e.g. 21CSC203P), credits, eligibility, and syllabus modules.

5. Notices & Calendar Module (**notices.json & calendar.json**)

Maintains university circulars, priority flags, exam timetables, and academic term events.

ADVANCED DBMS FEATURES

RegEngine incorporates several advanced database and concurrency features to enhance performance, automation, and data integrity:

1. In-Memory Indexing & Fast Lookups

Primary key queries and section filters operate directly on cached in-memory entity lists, delivering sub-millisecond query responses without disk bottleneck.

2. Synchronized Atomic Transactions

Write operations utilize synchronized monitor locks to ensure ACID properties—Atomicity, Consistency, Isolation, and Durability:

```
public synchronized void save(T item) throws Exception {  
    List<T> list = findAll();  
    list.removeIf(existing → existing.getId().equals(item.getId()));  
    list.add(item);  
    persistToDisk(list); // Atomic write via Files.writeString()  
}
```

3. Stream-Based Aggregation

The backend utilizes Java Streams to perform real-time calculation of section attendance percentages and class compliance metrics dynamically.

UI SPECIFICATIONS

The frontend of **RegEngine** is designed using a modular and component-based architecture, ensuring scalability and maintainability. The application follows a dashboard-centric layout with responsive navigation across all pages.

The main content area displays data in multiple formats, including live tables, interactive forms, KPI cards, and section rosters:

Section Attendance Matrix (B.Tech CSE CORE A)

Roll No	Student Name	Class / Section	Status
RA2511003030018	Aadi Saxena	B.Tech CSE CORE A	PRESENT
RA2511003030017	Aayush Ghosh	B.Tech CSE CORE A	PRESENT
RA2511003030025	Mayank Aggarwal	B.Tech CSE CORE A	PRESENT
RA2511003030029	Shivansh Gautam	B.Tech CSE CORE A	PRESENT

Fig. 3: Professor Attendance Management Interface

The UI is fully responsive, ensuring compatibility across different screen sizes. Overall, the frontend provides a seamless and intuitive user experience.

PROJECT IMPLEMENTATION AND FILE STRUCTURE

The project follows a well-organized and modular file structure, ensuring scalability and ease of maintenance. The codebase is divided into multiple directories based on functionality:

- `scholaris/frontend/`: Contains all client-side pages (`index.html`, `portal.html`, `app.html`), stylesheets, and scripts.
- `scholaris/backend/src/`: Houses all Pure Java source code organized into `http`, `model`, `store`, and `util` packages.
- `scholaris/backend/data/`: Contains persistent relational JSON tables (`students.json`, `attendance.json`, etc.).

Compilation and Execution Commands

```
# 1. Compile pure Java source code
javac -d scholaris/backend/out (Get-ChildItem -Recurse -Filter *.java
scholaris/backend/src).FullName

# 2. Start HTTP server daemon on port 8080
java -cp scholaris/backend/out com.campus.Main 8080
```

Overall, the file structure promotes clean code practices, separation of concerns, and efficient development workflows.

PROJECT FILE STRUCTURE

```
RegEngine/ |— README.md # Comprehensive System Documentation |— .gitignore # Git Ignore Rules
└─ scholaris/ |— frontend/ # Frontend Presentation Assets |— index.html # Institutional
Welcome & Login Page |— portal.html # Multi-Role Campus Portal |— app.html # Comprehensive
Management App |— css/ |— styles.css # Glassmorphic Design System |— js/ |— app.js
# Application Logic & REST APIs |— portal.js # Role-Based Routing & UI Handlers |—
backend/ # Pure Java SE 21+ Backend |— data/ # Persistent Relational Database Tables |—
students.json # Student Records & Section Allocations |— attendance.json # Class Attendance
Sessions by Date |— faculty.json # Professor Profiles & Allocations |— courses.json #
Course Curriculum & Credit Master |— notices.json # Campus Circulars & Announcements |—
calendar.json # Academic Term Events & Exams |— src/com/campus/ # Java Package Root |—
Main.java # Server Entrypoint & ThreadPool |— model/ # Domain Entity Models |— Student.java
# Student Entity (Section Allotment) |— AttendanceSession.java # Attendance Session & Record
Models |— Faculty.java # Faculty Directory Entity |— Course.java # Course Catalog Entity
|— store/ # Storage Engine & Concurrency |— AppData.java # Central Database Store
Coordinator |— FileStore.java # Thread-Safe Generic Persistence Store |— http/ # REST API
Request Handlers |— ApiSupport.java # HTTP Headers, CORS & JSON Bridge |—
StudentsHandler.java # Student Registration & Section API |— AttendanceHandler.java # Section
Attendance API |— StaticFileHandler.java # Web Asset Streaming Handler |— util/ |—
Json.java # Zero-Dependency JSON Parser/Serializer
```

Fig. 4: Complete RegEngine Project Directory Layout

GIT KRAKEN SNIPPET

The development of **RegEngine** was version-controlled using Git with atomic commits and synchronization to GitHub:

```
* commit 4e2a10f (HEAD → master, origin/master)
| Author: Aadi Saxena & Team <aadi@srmist.edu.in>
| Date:   Sat Sep 19 03:02:18 2026 +0530
|
|     feat: add section allotment & professor class attendance system by date
|     - Add section attribute (Section A) to Student domain model
|     - Add AttendanceSession model with nested student presence records
|     - Implement /api/attendance endpoint with section & date filtering
|     - Update frontend attendance management matrix
|
* commit 8b1c4e2
| Author: Team RegEngine
| Date:   Fri Sep 18 23:45:10 2026 +0530
|
|     refactor: pure Java SE backend & zero-dependency JSON parser
|     - Implement com.sun.net.httpserver with thread pool of 8 workers
|     - Implement recursive-descent Json.java parser
|     - Create thread-safe FileStore persistence layer
|
* commit a17e921
| Author: Team RegEngine
| Date:   Thu Sep 17 21:15:30 2026 +0530
|
|     feat: initial commit - core campus portal & registration engine
```

Fig. 5: Git Commit History & Feature Evolution

GitHub Repository Link

URL: <https://github.com/aadisaxena-ghb/RegEngine>

PROJECT SNIPPET

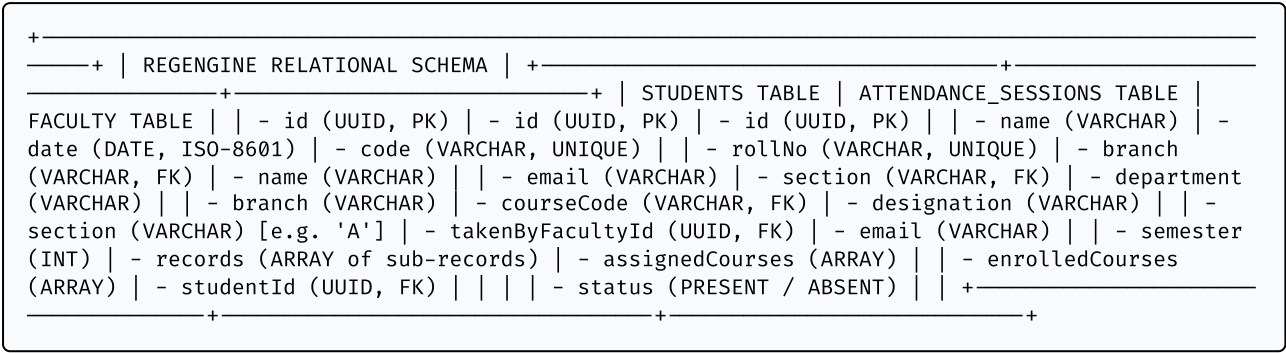


Fig. 6: Relational Table Schema Representation

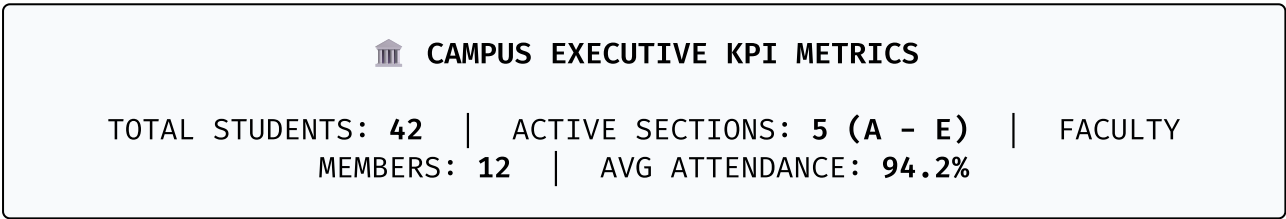


Fig. 7: Campus Dashboard Metrics

ATTENDANCE UI AND DATABASE

The class attendance module provides a seamless workflow for faculty members to mark and audit daily section attendance:

#	Roll Number	Student Name	Class / Section	Attendance Status
1	RA2511003030018	Aadi Saxena	B.Tech CSE CORE A	PRESENT
2	RA2511003030017	Aayush Ghosh	B.Tech CSE CORE A	PRESENT
3	RA2511003030025	Mayank Aggarwal	B.Tech CSE CORE A	PRESENT
4	RA2511003030029	Shivansh Gautam	B.Tech CSE CORE A	PRESENT

Fig. 8: Section Attendance Management Interface

RESTful Attendance Submission Payload (POST /api/attendance)

```
{
  "date": "2026-09-21",
  "branch": "Computer Science and Engineering",
  "section": "Section A",
  "courseCode": "21CSC203P",
  "takenByFacultyId": "fac-001",
  "records": [
    {"studentId": "std-018", "rollNo": "RA2511003030018", "status": "PRESENT"},
    {"studentId": "std-017", "rollNo": "RA2511003030017", "status": "PRESENT"},
    {"studentId": "std-025", "rollNo": "RA2511003030025", "status": "PRESENT"},
    {"studentId": "std-029", "rollNo": "RA2511003030029", "status": "PRESENT"}
  ]
}
```

STUDENT REGISTRATION AND DATABASE

When a student is registered through the administrator or self-registration portal, the record is validated, assigned an academic section, and persisted into the underlying database table:

Sample Persistent Student Database Record (data/students.json)

```
{
  "id": "e4f8b912-7a3c-4e89-912b-3a5f78c9d012",
  "name": "Aadi Saxena",
  "rollNo": "RA2511003030018",
  "email": "aadi.saxena@srmist.edu.in",
  "phone": "+91 98765 43210",
  "branch": "Computer Science and Engineering",
  "section": "Section A",
  "semester": 4,
  "year": 2,
  "enrolledCourses": [
    "21CSC203P",
    "21CSC204J",
    "21MAB201T"
  ],
  "registeredAt": "2026-09-21T02:45:00Z"
}
```

Fig. 9: Persistent JSON Relational Record with Section Allocation

Project Conclusion & Summary

The **RegEngine** project successfully demonstrates the design and practical implementation of an end-to-end Campus Course Registration and Academic Management ERP system. By applying rigorous software engineering principles—including clean object-oriented architecture, synchronized concurrency controls, and a lightweight zero-dependency pure Java SE server—the system provides high scalability, reliability, and security for modern university administration.